

Project 5: Physical Database Design and Tuning

Due date: 29 April 2026 before 9 p.m.

Recall from the previous projects, you already collected and analyzed database application requirements, designed a conceptual database model using an Entity-Relationship model, and mapped the conceptual database model to a logical database model represented in form of relational database schema. Then, you already went to the physical database design by implementing the relational database schema using the Structure Query Language (SQL) and a Relational Database Management System (RDBMS).

In this project, you will have the physical database design and go to the database tuning. You are required to analyze database application requirements to choose appropriate database access structures and view maintenance technique to help improve the performance of the query processing and transaction management. By following the previous projects, you have to continue your works individually.

Things to do

- Analyze database application requirements and choose at least **three transactions** (or queries) that are most important to the performance of your database application.
- Choose and implement appropriated **file organizations** and **index structures** that could help improve the performance of your database application. **Triggers** and **assertions** are optional.
- Perform experiments to compare the query processing performance of each transaction (or query) before and after implementing the appropriated file organizations and indexes structure. You may also load a large number of data records into relations that related to the selected transaction so that you can see the improvement.

Discuss the result of the experiments. The RDBMS's query analyzer can be used to help analysis of the experiments.

Concerned Issues

1. Plagiarism is NOT allowed for any cases.
2. Creativity in design and presentation is preferable.

Deliverables

A document in portable document format (PDF). The content in the document includes:

1. The *database application requirements* including description of required transactions (or queries) with **highlighted** the selected transactions (or queries).
2. The final *conceptual database model* (i.e., ER diagram) with **highlighted** the portion that related to the selected transactions (or queries).
3. The final *logical database model* (i.e., Relational database schema) with **highlighted** the portion that related to the selected transactions (or queries)
4. The *SQL commands* only related to the selected transactions (or queries) and the specification of the related table, e.g., the number records in the table, the record sizes.
5. The results of the analysis of each selected transaction before improvement. The result from RDBMS's query analyzer can also be attached (as a figure).
6. The results of the analysis of each selected transaction after improvement. The result from RDBMS's query analyzer can also be attached (as a figure).

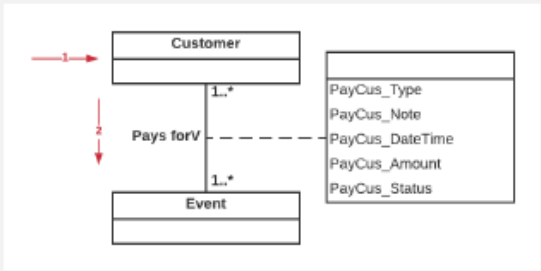
Faculty of Information and Communication Technology
ITCS 442 Relational Database Design

7. The discussion on the results.
8. References (if any).

Submissions

1. If there are more than one file to be submitted, zip them. Then, name the file as <ProjectID><GID><last-three-digit-of-StudentID for every team member>.pdf. For example, "P5_G01_999_988_977.zip".
2. Submit the file on MyCourses: <https://mycourses.ict.mahidol.ac.th/>

Samples of transaction analysis:

Transaction Analysis Form					
					April 1, 2021
Transaction	(c) Update/ delete the details of payment status.				
Volume	Average:	1 per hour			
	Peak:	10 per hour (between 12.00-13.00 Monday-Friday)			
<pre> UPDATE Pay_for SET Pay_for.PayCus_Status = 'Complete' WHERE PayCus_Cus_ID = 'CUS317656' AND PayCus_Event_ID = 'EVT326668' -----or CREATE PROCEDURE UpdatedPayStatus_Cus @PayCus_Status NVARCHAR(20), @PayCus_Event_ID NVARCHAR(9), @PayCus_Cus_ID NVARCHAR(9) AS BEGIN SET NOCOUNT ON; UPDATE Pay_for SET PayCus_Status = @PayCus_Status WHERE PayCus_Event_ID = @PayCus_Event_ID AND PayCus_Cus_ID = @PayCus_Cus_ID END EXEC UpdatedPayStatus_Cus @PayCus_Status = 'Complete', @PayCus_Event_ID = 'EVT326668', @PayCus_Cus_ID = 'CUS317656' GO </pre>		Predicate: PayCus_Cus_ID = 'CUS317656', PayCus_Event_ID = 'EVT326668' (in this case, we assume we know the customer ID and eventID, but instead, the customer's name and event name is also possible)			
		Join attributes:	-		
		Ordering attribute:	-		
		Grouping attribute:	-		
		Built-in functions:	-		
		Attributes updated:	PayCus_Status		
					
Access	Entity	Type of Access	No. of References		
			Per Transaction	Per Transaction	Peak Per Hour
1	Customer	R	1000	1000	10000
2	Pay_for	R	7000	7000	70000
3	Pay_for	U	1	1	10
4	Event	R	200	200	2000
Total References			8201	8201	82010